

Unveiling the Nexus: Harnessing IoT Ecosystems for Internet Censorship Evasion

Anonymous Author(s)

Abstract

In recent years, numerous research breakthroughs have emerged in the realm of anti-censorship, including automated methods to probe and generate censorship circumvention policies, network packet manipulation, and cryptography. Among the practical implementations of circumvention systems, centralized design prevails as the most popular approach, which mainly utilizes a single routing entity (servers, data centers, etc.) with predetermined IP addresses and cryptographic protocols. However, despite popularity, centralized circumvention models fall short of ad hoc censorship systems, meaning that such systems are often designed to target one specific country (China, Turkmenistan, Russia, etc.). Such adversaries enforce Internet blockade through mass IP range blocking and explicitly designed detection heuristics targeting protocols known to be mainly used for censorship circumvention. In response, we introduce a new approach with *Nexus Packet Fragmentation Algorithm* as the first evaluated step towards *Nexus*, our proposed circumvention model based on a low-cost, semi-autonomous IoT-based distributed design without the need for centralized servers for coordination or routing. We present our algorithm as the core of our future design to offload routing and obfuscation tasks to dispersed IoT devices across networks, which possess computational power and vast IP spaces in unison. At the current stage, we show that our algorithm bypasses censorship systems by sending packets in dissected fragments with lightweight algorithms based on bit-manipulations, enforcing entropy patterns, and smoothing packet lengths. We tested the rudimentary fragmentation algorithm on our testbed in China and found that the algorithm successfully enabled circumvention through the Great Firewall (GFW) in China, accessing 100% of blocked websites and resources listed by the Citizen Lab at 11,453.5255 bytes per second, even in the absence of our proposed distributed network and deployed on one low-power IoT device. We demonstrate that the low-cost IoT network approach is valid against the current state-of-the-art of censorship techniques. In the end, the fragmentation algorithm serves as the stepping stone towards implementing the full distributed design of *Nexus*, offering a new path forward to address the inefficiencies associated with both distributed and centralized circumvention approaches.

Keywords

IoT, censorship, distributed network

1 Introduction

Internet censorship has been a prevalent and ever-growing issue around the world, now not limited to those countries nominally known for such practices, including China, Iran, and Russia [4, 5, 39], even among countries that are perceived to enforce little Internet censorship [31], increasing the urgency of the matter. Across the globe, Russia and China are prominent censorship actors, implementing censorship through a wide range of IP blocking, DNS pollution, or dropping unauthorized connections to prohibited services detected via fingerprinting censorship-specific protocol [15, 35, 39]. In addition, although much research has been published throughout the past few years, it was mostly composed of exposing vulnerabilities within censorship systems—such as the GFW of China—that can be exploited for circumvention [13, 35], but fell short in offering a new direction to exploit these novel findings in practice.

Incidentally, the rapid application of Internet of Things (IoT) devices offers valuable means for scientific testbeds, data collection, and data storage due to their low cost, low power consumption, and billions of IP addresses around the world [2, 8]. This paves a new pathway to a volunteer-based, distributed, and self-driven network for censorship circumvention routing without any centralized or static servers.

Hence, this paper combines the two subjects of censorship circumvention and IoT-based distributed systems to devise a new method, deviating from traditional mimicry-based circumvention techniques [16].

We propose *Nexus*, a new censorship circumvention model based on a distributed IoT network, and introduce the *Nexus Packet Fragmentation Algorithm* as the initial touchstone to validate the foundations. With our design composed of an IoT-based distributed network, *Nexus* could potentially hinder Deep Packet Inspection (DPI), Device Fingerprinting (DFP), and other traffic-correlation analysis sanctioned by nation-states [35] by fragmenting packets into separate packets and sending them across different routes, each containing only parts of the original whole packet. This can be achieved at a fraction of the cost and show improved performance (with parallelism) compared to preexisting distributed circumvention models, thanks to the economical nature of IoT devices and therefore vast deployment across IP ranges. This will be explained in Section 3.1.

We hypothesize possible attack models based on a comprehensive review of previous work ranging from ML classifiers deployed to detect high-throughput obfuscated traffic [21, 24] to known techniques (e.g., active probing and deep-learning ML models) utilized by nation-states to fingerprint possible VPN tunnels [32, 40]. With our example algorithm for the worst case scenario where there is no encryption on the payload at all, it fragments plain-text packets by sensitive keywords at byte level, pads each packet to flatten the IoT fingerprints—most notably packet length and stream direction [7, 22, 24, 30]. *Nexus* is designed to circumvent the heuristics

This work is licensed under the Creative Commons Attribution 4.0 International License. To view a copy of this license visit <https://creativecommons.org/licenses/by/4.0/> or send a letter to Creative Commons, PO Box 1866, Mountain View, CA 94042, USA.

Proceedings on Privacy Enhancing Technologies YYYY(X), 1–9
© YYYY Copyright held by the owner/author(s).
<https://doi.org/XXXXXXX.XXXXXXX>



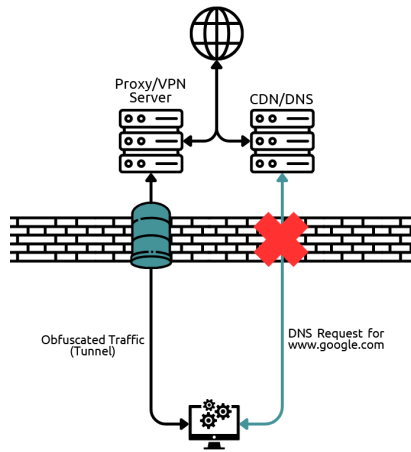


Figure 1: Traditional VPN Approach

implemented by the GFW [35], and sends the fragments through a network of IoT devices in parallel via a Distributed Hash Table (DHT), with minimal encryption (unless otherwise specified) applied to the custom metadata header.

2 Background

Traditionally, centralized VPN services have been the predominant means of Internet circumvention, mainly through popular protocols such as *OpenVPN* and *WireGuard* for ease-of-use, customizability, and robustness [40]. Alternatively, *Tor* presents one of the only general-use decentralized circumvention solutions, which routes clients' Internet traffic through volunteers' PCs through multiple hops to reach the destination [9]. While both methods have experienced varying successes in combating state-sanctioned censorship and making themselves ubiquitous to the general public through open-source, they have critical vulnerabilities in some aspects.

2.1 Centralized Approaches

Although mainstream circumvention protocols are competent in payload obfuscation and simplistic implementations [10, 17], they are prone to fingerprinting, active probing, and detection-by-heuristics [35].

Per previous works by Diwen Xue et al., VPN services utilizing *OpenVPN* are subject to active probing from adversaries by fingerprinting their handshake procedures and responses from specifically constructed payloads [40]. Furthermore, *WireGuard*, a VPN protocol designed for lightweight use, does not focus on obfuscation and is known to be easily detected by DPI due to its consistent fingerprints from the headers [10, 34], allowing the adversary to block the protocol en masse without having to acquire knowledge about the plaintext payload. Given the vulnerability of centralized circumvention designs to targeted blocking and fingerprinting, some researchers have turned to other distributed approaches instead.

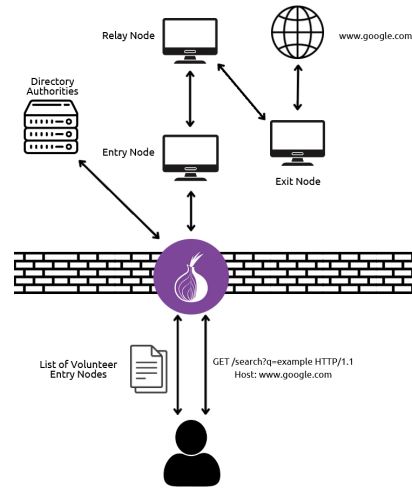


Figure 2: Simplified overview of Tor's Infrastructure

2.2 Distributed Approaches

In contrast to the aforementioned centralized circumvention architectures, *Tor*'s design assigns volunteer nodes ("*Tor Bridges*") through a coordinator index server with corresponding keys to ensure secret transmission of the packets via many pluggable transport protocols, including *obfs4* [9]. However, while *Tor* offers a distributed architecture that is challenging for censors to detect or decrypt the packets due to its powerful protocols and distributed design, it poses constraints on performance compatibility with certain volunteer devices and raises certain privacy concerns for the volunteers. To address these issues, the *Tor Project* team also introduced the *Snowflake* architecture, allowing more users to participate (with ephemeral presence in the system) by just using designated browsers embedded with a more lightweight peer-to-peer WebRTC protocol and forwarding traffic to a centralized bridge [6].

However, the persistent existence of a centralized index server in both iterations implies a single point of failure in the entire system, even though it is not used for routing. Although this can be mitigated by distributing the relay list through domain fronting, with the diminishing number of Content Delivery Network (CDN) services that allow domain fronting at all, the concern remains. In addition, the current design of *Tor Bridge* and *Snowflake* poses potential risks to the client in particular, but plausible circumstances. Should the adversary impersonate as an authentic volunteer and gain the role of entry node, allowing them to inspect the source client's IP, the state-sanctioned adversary has the potential to impose physical threats on the client. Additionally, the adversary impersonating as an exit node would also enable them to intercept unencrypted data. Such risks would also be amplified if both ends of the traffic (entry and exit nodes) are under the said adversary's control. Additionally, although *Snowflake* proved to be user-friendly and a step-up from the original *Tor Bridges* in terms of ease of usage, it had yet to address deployability on low-cost, low-power devices.

Therefore, inspired by the colossal number of IoT devices in the wild, which occupy a much wider IP space [2, 8], the proposed

Nexus aims to improve the resolutions to these issues by, instead, letting a network of semi-autonomous IoT devices collectively act as index servers, whilst transmitting small, fragmented chunks of data packets among each volunteer IoT device with lightweight to no encryption, thus lightening the burden of receiving large chunks of data and decryption done on the volunteer device. As we will explain in Section 5, not only does the lack of encryption not significantly increase the said risks, but it also, albeit theoretically, provides better performance if the transmissions of the fragmented packets are done in parallel.

3 Motivation

Despite being the major core of circumvention throughout the past decades, circumvention-by-mimicry has been argued to be ineffective against massively funded censorship infrastructures [16, 35, 40]. In their previous work, Amir Houmansadr et al. stated that mimicry-based circumvention methods are unsustainable against state-sanctioned censorship heuristics, implying that future circumvention models should diverge from developing circumvention techniques that impersonate traffic as regular services or mere padding schemes [16]. Therefore, we provide *Nexus* as a distributed alternative, focusing on traffic flow manipulation.

In addition, although distributed censorship circumvention models, such as *Tor*, have existed for a while, our model aims to obtain improved cost and scalability with cheaper IoT devices.

3.1 Cost

With designs such as *Tor*'s, it is assumed that the volunteers would deploy the *Tor Bridge* on a Personal Computer (PC) to serve as a "jumping server" to circumvent censorship without going through centralized servers with static IP(s) [9]. While robust and proven to function sufficiently over the decade, its computation-heavy protocol scheme (e.g., *obfs4*) requires at least somewhat capable devices such as a laptop to run on, which can cost from \$100 up to \$2,000 in USD [1]. In contrast, a commonly used System-on-Chip (SoC)—a type of bare-bone IoT device—with Wi-Fi capability, namely the ESP32 SoC, would only cost single digit in USD [29]. Although less capable of handling large traffic clusters and computationally heavy encryption functions due to bandwidth and CPU limitations, we aim to reach equal routing performance with a fraction of the cost through *Nexus*.

3.2 Scalability

With the drastic decrease in monetary cost per unit, full utilization of IPv6 space becomes much more attainable. With tens of billions of low-power IoT already in the wild and occupying IPv6 addresses [2], *Nexus* employs the new-found vast network of nodes to dampen the most commonly used general-use network traffic fingerprinting features—particularly traffic direction [7, 22, 24, 30]—by dissecting the network packets into fragments and simultaneously routing them to distinct volunteer IoT nodes within the distributed network. Such scalability would hinder censors' ability to identify and drop prohibited actions non-trivially without needing fully encrypted channels, and also be compatible with encryption when the delivered content itself requires high confidentiality.

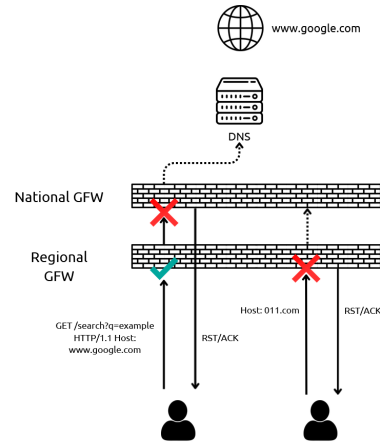


Figure 3: Simplified Demonstration of the Regional/National GFW [36]

In addition, as discussed in Section 6.1, in contrast to the design of *Tor*, this paper also proposes to implement *Nexus* on a DHT-based mechanism to distribute a partial list of "neighbor" nodes in lieu of index servers for the coordination of volunteer nodes with minimal potential damage by adversaries posing as volunteers, as no one node would have the full topology of the network.

4 Adversary Model

The adversary we consider in this work has comprehensive control over their regional deployment, censoring through ISP-level on-path DPI (looking for plain keywords or fingerprints) and DFP, IP range blocking, etc [3, 11, 15, 33, 38]. Specifically, we presume that the GFW is implemented on critical Internet infrastructures, such as the Border Gateways of Autonomous Systems that connect the Internet in China to the outside world [27]. The most common symptoms that indicates an occurrence of blocking would be TCP RST attacks and consistent packet drops [33]; we also expect that an adversary may be able to trace the real identity through mere source IP addresses.

In contrast, we also assume that some parts of GFW are regionally deployed [36], and coordination among these regions is relatively infrequent. This implies that it might be implausible for the system to identify that the corresponding fragmented packets are incidentally routed through Autonomous Systems (ASes) in different regions. Because of this, we also assume that these regional adversaries within China do not necessarily have identical or similar Standard Operating Procedures (SOPs). This loophole presents a potential vulnerability in on-path network surveillance, as provincial-level censorship systems might have different levels of risk-aversion and definitions of suspicious network traffic [36].

Moreover, according to many fingerprinting techniques developed by previous work, mere stream direction (upstream or downstream, and ports connected), packet lengths, and timings would be sufficient to identify traffic of interest without the need to decrypt anything [7, 22, 24, 30, 35]. Thus, we assume that the adversary

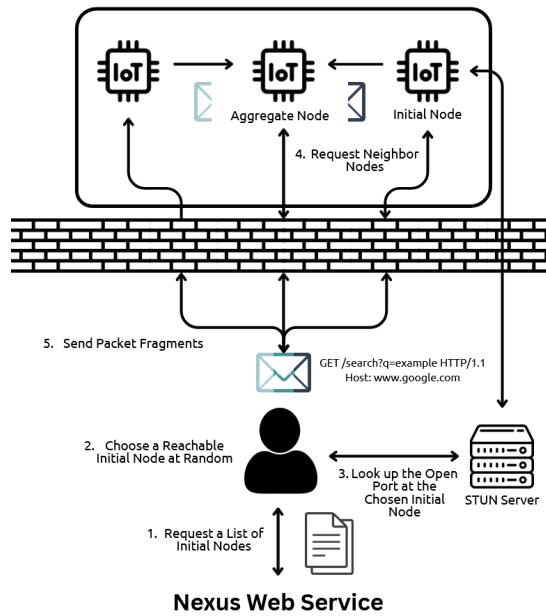


Figure 4: Communication between Client and Nodes

would implement similar techniques to detect “irregular” traffic from or to non-traditional devices that would not receive or initiate connections across the border.

Lastly, with regard to the concerns surrounding Quantum Computing, many projects have also integrated quantum-resistant algorithms into their protocols preemptively [37]. However, due to a lack of evidence and reports for any deployment of the aforementioned tools or usage of Quantum Computers for censorship, such scenarios are not included in our adversary model.

Therefore, weighing the prominence and novelty factors of the censorship methods discovered, for this paper, we primarily focus on the adversary model in which the censor implements the threat model based mainly on those described in the papers by Wu et al [35, 36].

5 Nexus

Nexus is composed of a distributed network mainly of low-power IoT devices while remaining compatible with regular servers or high-computing devices. *Nexus* consists of two components: the fragmentation algorithm and the proposed distributed network of nodes, which will transmit and reassemble those fragments. We aim to meaningfully hamper real-time DPI and fingerprinting capabilities with this design, even if the encryption is only applied to the metadata.

We show the general workflow of *Nexus* in Figure 4. In the design, the client will fragment the original packet into an arbitrary number of pieces with our fragmentation algorithm and send them to different volunteer IoT devices. The volunteer nodes will decrypt

the metadata in each fragment, discover the peer nodes that received the other corresponding pieces. The nodes will then elect an aggregate node amongst themselves through a pseudo-Paxos consensus algorithm [20], and send the reassembled packet to the destination. The same fragmentation procedure will occur from the aggregate node, distributed with the aforementioned nodes, and send the fragments to the client’s device for reassembly of the response data.

5.1 Types of Volunteer Node Devices

As *Nexus* is designed to maximize the effectiveness of a distributed circumvention approach, we make node-side *Nexus* deployable on a wide range of devices. This includes low-cost IoT such as the ESP8266 and ESP32 SoCs often seen in Smart City equipment (such as smart trash cans, traffic lights, sensors, etc.) and consumer-grade Smart Home devices such as Amazon Echo and Fire TV. In addition, we also consider open-source projects such as *ESPHome* [12], which allows seamless integration of customized apps to be installed on devices running on ESP8266 or ESP32.

We categorize means of deployment into three levels, ranked by ease of deployment, from easiest to hardest:

- (1) Side-loading *Nexus* on Commercial Devices
- (2) Integrating Through Existing Platforms
- (3) *Nexus* as Firmware

Side-loading *Nexus* on Commercial Devices The most intuitive place to deploy *Nexus* for the average users would be the Smart Home IoT devices they have purchased. However, this implies that volunteers expect the devices to function as intended while running *Nexus* in the background. Therefore, side-loading *Nexus*—acquiring it without going through the respective official application distributor—would be the best means of installation.

To accommodate, *Nexus* is developed in Android, the core of many customized operating systems of mainstream commercial IoT devices, such as *FireOS*, *WearOS*, etc. While not universally guaranteed to be compatible, we make sure *Nexus* reaches the average group of IoT consumers, who occupy the majority of the market [2].

Integrating Through Existing Platforms When the cost of consumer-grade IoT devices goes beyond the budget, the next efficient solution would be to make use of existing platforms or systems designed for specific SoCs that allow modules to be installed. For instance, the aforementioned *ESPHome* allows *Nexus* to be installed as “Components”, dedicated to a specific functionality in parallel to others. With platforms such as this, *Nexus* can be installed on low-power IoT devices with relative ease and high compatibility.

***Nexus* as Firmware** For experienced volunteers, they have the choice to flash the entire device with the corresponding *Nexus* designated for a specific series of SoCs or architecture. Although low-cost, it requires volunteers to possess a certain level of expertise in embedded programming.

In general, while not inherently difficult, this will nullify the original purpose of the device, whether it be a sensor with real-time data transmission or a surveillance camera installed in front of the door. Therefore, this means of installation is the most complicated

and inconvenient for the average user.

5.2 Nexus Packet Fragmentation Algorithm

Algorithm 1: FragmentationByKeywords

Input: Packet: byte sequence, Keywords: list of byte sequences
Output: List of fragments after splitting the Packet

```

foreach keyword in Keywords do
    index ← position of keyword in Packet;
    if index is found then
        chosenKeyword ← keyword;
        break;
if no keyword found then
    return {Packet}; // Return packet as a single fragment
else
    pivot ← index +  $\left\lfloor \frac{\text{length}(\text{chosenKeyword})}{2} \right\rfloor$ ;
    LeftFragment ← substring of Packet from start to pivot;
    RightFragment ← substring of Packet from pivot to end;
    return FragmentationByKeywords(LeftFragment,
        Keywords) concatenated with
        FragmentationByKeywords(RightFragment,
        Keywords);

```

Algorithm 2: FragmentationByUniformRandomLength

Input: Packet: byte sequence, MinLen: minimum fragment length, MaxLen: maximum fragment length
Output: List of fragments after splitting the Packet

```

fragments ← empty list;
while Packet is not empty do
    randomLen ← UniformRandom(MinLen, MaxLen);
    if randomLen > length(Packet) then
        randomLen ← length(Packet);
    fragment ← first randomLen bytes of Packet;
    Append fragment to fragments;
    Remove the first randomLen bytes from Packet;
return fragments;

```

As the current iteration of Nexus is designated for censorship circumvention purposes with the worst case scenario in mind (no encryption on the payload), and there is a finite amount of known sensitive keywords and websites prone to censorship [28], we may utilize either the existing list or a uniform-random length for each

fragment to define our delimiters with Algorithm 1 and Algorithm 2 as the baseline naive approach.

Even with the example fragmentation algorithm of a naive keyword-based delineator, we present a computationally complex model that obstructs adversaries from fingerprinting traffic flow and enables the process to reassemble the original packets with the intercepted packets' detection hardness. Section 8 further discusses the matter of formally verifying such hardness using *Indistinguishability Proof* [25].

In addition, the algorithm may also be effective against on-path device fingerprinting, as it creates more unpredictable packet lengths and stream directions with semi-randomized fragmentation and multiple nodes for routing. We will demonstrate this in Section 7.1.

5.3 Metadata Header

During the handshake, the nodes respond to the client with an RSA public key, with which the client shares the AES-256 key and the initialization vector used to encrypt the protocol metadata, constructed as below:

$$\text{metadata}_{i,j} = \text{bytes}(\text{sessionID} + \text{totalNum}_{i,j} + \text{orderIndex}_{i,j} + \text{padNum}_{i,j} + \text{nodeList}_i) \quad (1)$$

$$|\text{metadata}_{i,j}| = L_{\text{metadata}} \geq 11 \text{ bytes}$$

$$\text{sessionID} = i \in \{P_i\}$$

$$\text{totalNum}_{i,j} = |\{PF_{i,j}\}|$$

$$\text{orderIndex}_{i,j} = j \in \{PF_{i,j}\}$$

$$\text{nodeList}_i = \{N\}$$

We denote the original unfragmented packets as P and fragmented packets of each P_i as $\{PF_{i,j}\}$. L_{metadata} is the byte length of the metadata header of $PF_{i,j}$, the j -th fragment of the packet P_i . Subsequently, the sessionID is the unique ID for the packet P_i , $\text{padNum}_{i,j}$ is the byte length of the padding in fragment $PF_{i,j}$, and $\text{nodeList}_{i,j}$ contains the list of nodes N assigned to P_i which receives and relays fragments $\{PF_{i,j}\}$.

With the design, we allow the routing nodes to acquire the necessary information to coordinate with each other and reassemble the packets while maintaining secrecy against on-path adversaries.

6 Proposed IoT-based Distributed Network

As part of Nexus' design, we also propose a distributed network designed to both distribute relay node lists and route networks. Albeit not tested in Section 7 due to resource constraints, we hypothesize that this would impede censors' ability to detect circumvention traffic and discover the ordering of corresponding packets, as shown in our results in Section 7.1.

6.1 Node Coordination

As a crucial part of the infrastructure, volunteer nodes must have the means to communicate with each other and share information regarding ongoing packet sessions. To do this, we propose

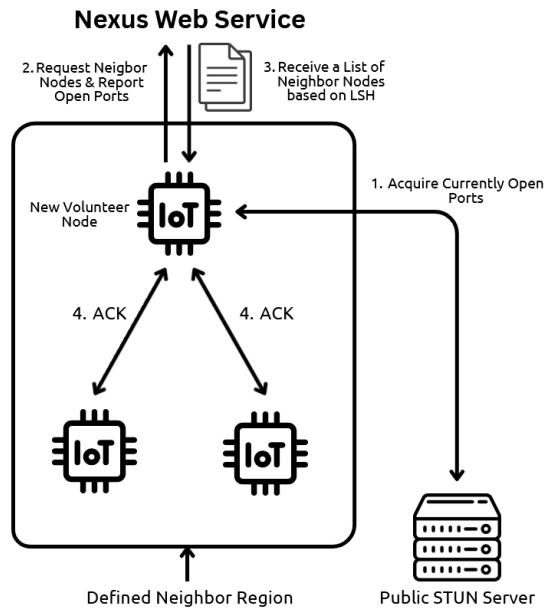


Figure 5: Bootstrapping Process of a New Volunteer Node

to implement a version of Distributed Hash Table (DHT) with quasi-Kademlia—a classical, efficient distributed storage architecture [23]—to share a list composed only of proximate nodes to which the user and volunteers can connect. The proximity can be defined by geographic distance, Euclidean distance based on encoding, or any arbitrary custom rules. This will allow us to minimize the implementation of a centralized connection list distribution system and attempt to minimize the risk of an impersonator (i.e., sensors pretending to be a legitimate volunteer) acquiring the entire topology of the *Nexus* network.

Volunteer Node Bootstrapping In the design of *Snowflake* by the *Tor Project*, it allows less tech-savvy volunteers to participate via Network Address Translation (NAT) Hole Punching [6], a technique that enables outside connections to go through the volunteers’ node devices behind routers’ NAT by initiating an outbound connection to the entry nodes with open ports first. While it lowers the technical barrier for the volunteers, the discrepancy in port-openness (whether the volunteer has their router port forwarded) implies that such a system requires “broker” servers to administer port-forwarded entry or exit nodes to send outbound connections to, decreasing the distributability.

For *Nexus*, we propose that all volunteer nodes serve as “brokers.” Shown in Figure 5, for a volunteer outside the censored region, the volunteer’s device shall bootstrap by first connecting to a predetermined list of Public STUN servers and acquiring its currently open port. After which, the volunteer will contact the *Nexus*’ distribution server, denoted as the Nexus Web Service (NWS), send the IP and port information along with a request for a list of IP addresses and open ports of neighbor nodes based on the distance defined by Locality-Sensitive Hashing (LSH) algorithms [18]. After receiving

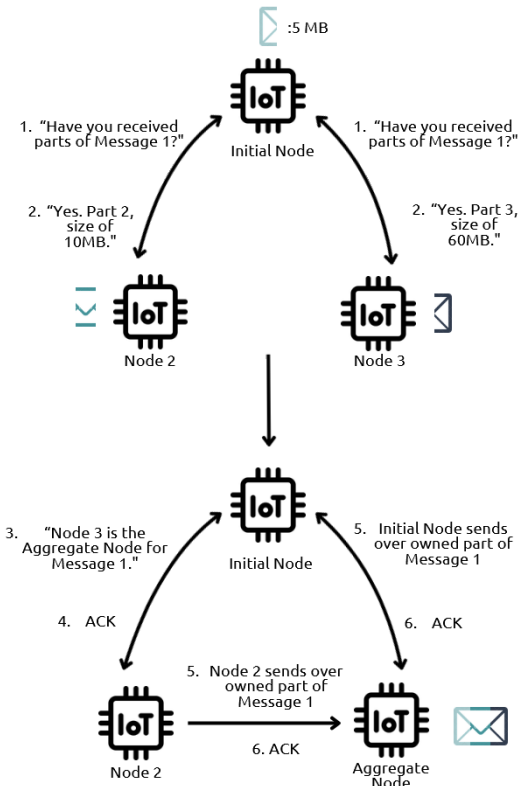


Figure 6: Inter-node Communication & Aggregate Node Election

such a list, the volunteer attempts to establish handshakes with the neighboring nodes. Upon success, these nodes will form a neighbor region, with the bootstrapped node responsible for receiving initial communications from the client users. By doing so, even if a node previously connected to the network loses connection to the NWS, it would still be able to receive the updated peer node list through its current peers.

Client-Node Communication & File Reassembly As illustrated in Figure 4, when a client user installs *Nexus*, a list of potential initial nodes will be provided by the NWS for initiating handshakes, which will be updated in the case where the number of unreachable nodes exceeds a certain percentage threshold on the list. The user will then search for the IP address and look up the open port in sequence on the predetermined STUN servers, halting at the first reachable initial node. After the initial connection, the user will receive the list of neighbor nodes, accepting fragmented packets from the user. The process will repeat for every new connection established.

As demonstrated in Figure 6, with the aforementioned metadata structure, shown in Equation 1, the nodes will decide the aggregate node based on file size or an arbitrarily defined process. The aggregate node will then accumulate all fragments, reassemble the

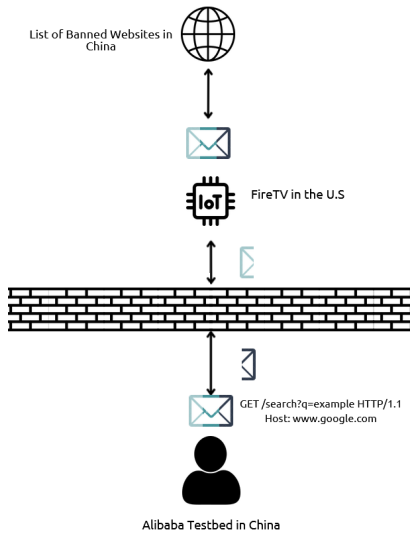


Figure 7: Testing Topology

original packet, and then be responsible for the downstream and upstream entry point between the final destination server and the client user. This stream of connection will persist over the course of the user’s continuous connection until actively terminated.

7 Test Designs

Shown in Figure 7, we design our testing environment as close to a real-world setting as possible. We deployed the client on an Alibaba Cloud server located in China, simulating a user within the censored regions.

In addition, to account for the lowest security bound due to computational constraints, no encryption was implemented other than on the metadata. We utilize two sets of test procedures:

Continuous Ephemeral: We make 10000 continuous requests to censored websites in China listed by the Citizen Lab [19] within 24 hours.

Intermittent Extended: We make 100 requests to censored websites in China listed by the Citizen Lab [19] every 24 hours, repeated continuously for 3 days.

The purposes of the two distinct tests are to

- (1) Verify the feasibility of *Nexus Packet Fragmentation Algorithm*, thus setting a benchmark for the full implementation of *Nexus*.
- (2) Test GFW’s ability to detect any fingerprint and categorize them as an anomaly within a short time, given a relatively high number of samples.
- (3) Test GFW’s ability to detect any fingerprint and categorize them as an anomaly within an extended time, given a small number of samples.

We utilize a FireTV Stick by Amazon to simulate a possible volunteer node, which was located in our testing location in the United States, as demonstrated in Figure 7. We chose FireTV for its low cost, its popularity among the average consumer, and its computational constraints. These features allow us to verify the viability of the

Table 1: Continuous Ephemeral (Upstream, Fire TV)

Attributes	In Bytes
Average Packet Length	411.64
Median Packet Length	66
Packet Length Coefficient of Variation	2.175
Shannon Entropy	1.7152
Throughput	11,453.5255/s

Table 2: Intermittent Extended (Upstream, Fire TV)

Attributes	In Bytes
Average Packet Length	430.61
Median Packet Length	66
Packet Length Coefficient of Variation	2.188
Shannon Entropy	1.5539

fragmentation algorithm as a baseline to *Nexus*’ general usability when deployed in a real home environment and its performance with a low-power CPU, which could not handle *Tor*’s protocol schemes [9] as consistently as regular PC platforms.

7.1 Results

Fingerprinting As shown in Table 1 and Table 2, with the fragmentation algorithm, we have achieved a high coefficient of variation (CV) in packet length to account for the commonly deployed length-based fingerprinting techniques and a Shannon Entropy below the blocking threshold of the GFW discovered in previous works [35]. We presume that a less consistent pattern of packet directions will also be observed should the multi-node design be properly implemented.

The algorithm enabled successful circumventions with seemingly no RST or packet drop actions by the GFW, despite only using one IoT node for routing with minimal encryption. This implies that it is effective against the predominant fingerprinting techniques that combine both packet directions and packet lengths as critical classification features [7, 22, 24, 30].

Downstream Flow Surveillance Behavior During the test, we also observed that there was no sign of blocking against the inbound traffic if it is a response within the same TCP session, even when the response file is unencrypted and originates from a prohibited website. This finding aligns with the hypothesis by Houmansadr et al [35].

Although, unfortunately, we were unable to implement this test on the aforementioned distributed system design in Section 6 due to resource constraints, we find the fragmentation algorithm effective in bypassing the GFW consistently without the common behaviors indicating blocking (e.g., RST response, packet drop, etc.), and thus offers propitious foundations for implementing the overarching design of *Nexus*’ the distributed system in the future.

8 Discussions

Despite initial successes, we recognize the following limitations in the design of the current prototype and test environments, elucidating potential areas for improvement in the future.

Test Environments for Nexus’ Distributed System Due to resource constraints, we could not test the DHT functionality on our current prototype. Therefore, although achieving successes via a single consumer-grade IoT, the proposed DHT-based coordination system and the hypothetical improved network speed with parallel transmission within the *Nexus* network remain theoretical, leaving room for future implementation tests as it matures with more public support from the open-source technical communities.

Impersonator Nodes Although *Nexus* uses the DHT in place of index servers to coordinate the volunteer nodes to mitigate the threat of impersonator nodes, it was not eliminated completely. While *Tor* protects the clients against malicious volunteer nodes with *Entry Guards*, it only decreases the clients’ chance of being profiled but does not conceal the fact that the clients attempted to connect to the *Tor* network; it also does not prevent intermediary nodes from being discovered and consequently blocked.

Similarly, *Nexus* passively protects the client through the combination of fragmentation and DHT, ensuring that no one single group of nodes will ever acquire all packet fragments or the entire topology of the *Nexus* network. However, this is still a play of probability in the hope that the clients will not connect to the malicious nodes with no guaranty, particularly when *Nexus* is designed with a self-managed network in mind.

Indistinguishability Proof While the test results of *Nexus*’ fragmentation algorithm achieved a promising start, we do not yet have the proper verification formula or toolkit to formally verify its mathematical complexity. This exposes a long-lasting vulnerability in censorship research at the theoretical level, requiring recurrent trial-and-error processes along the way. Building upon this, future research may focus on general-use methodologies and procedures to formulate proof of undetectability and indistinguishability for newly developed circumvention models. For instance, the *VeriWeird* project provided a novel approach coined as *simulatable indistinguishability proofs* (SIPs), which formalizes covert communication channels based on the encoding of their properties in the “simulation-based cryptographic proof paradigm.” [25] Such a formalization would not only accelerate future censorship research projects without compromising reliability, but would also help technical communities to rigorously verify the feasibility of a novel approach.

9 Ethics

9.1 Volunteer & User Privacy

This project respects the privacy of all potential beneficiaries. Therefore, we did not solicit third-party participation in our testing process. We also inform potential interested developers about possible breaches of personal information (such as IP addresses) from both the clients and the volunteers, as we have yet to discover a method to guarantee the prevention of potentially malicious nodes. This

issue is much more pronounced when the effectiveness in disrupting traffic flow fingerprinting of *Nexus* could be improved, should there be intermediary nodes within the censored region, exposing volunteers of these nodes to potential repercussions.

Furthermore, without a robust validation system to prevent censors from posing as legitimate volunteers, the risk of exposing client users’ information persists as it does for *Tor* [26]. As such, there exists a possibility that a user may happen to connect to an impersonator Initial Node, which can also be selected as an Aggregate Node. Although low, the possibility alone can lead client users to legal issues, accusations, or even incarceration within censored regions [14]. The severity of the risk compels the ethical obligation to deter the public from implementing the model of the paper without any alteration, and to be wary of possible consequences.

However, as mentioned in Section 8, one path to alleviate the concern is to implement a trust-ranking system based on features that do not fully break the anonymity of volunteers (e.g., IP address reputation, length of valid volunteering time, etc.). Such a system would discourage censors from devoting resources to deploy malicious nodes by rendering the method inefficient, whilst not drastically degrading the principles of the distributed network by giving complete administration over the Initial Nodes and Aggregate Nodes to a single entity (i.e., *Nexus*’ official team). At this time, the proposed *Nexus* model reduces risks for volunteers with a node assignment distribution mechanism, where no one volunteer node will gain a list of nodes for the entire topology given a specific session, therefore limiting the effectiveness of impersonator nodes. Nevertheless, it is still inadvisable to deploy *Nexus* as a participant within the censored region.

9.2 Use of IP addresses from Public Spaces

We acknowledge that *Nexus* might be deployed in public spaces accompanied by hole-punching nodes, as IoT devices are often covert, and a public network could reduce the possible risk of personal identity breaches. The authors discourage such actions due to the aforementioned risk involved, as “innocent bystanders”—i.e., the owners of such public places— will face possible legal liability unwillingly and unknowingly. Therefore, we ask that interested parties refrain from acting in this capacity unless supported by the proprietor.

10 Conclusions

In this paper, we present the *Nexus Packet Fragmentation Algorithm* as the first validated component of *Nexus*, a proposed distributed IoT-based circumvention system. We evaluated the algorithm and showed initial success, building a promising foundation. With the prototype, we propose a new distributed approach for future censorship research, eventually moving away from mimicry-based obfuscation, instead towards increasing computational burdens for on-path censors in both packet inspection and real-time traffic surveillance.

References

- [1] Talha Barkaat Ahmad. 2024. Laptop Prices Dataset - October 2023. <https://www.kaggle.com/datasets/talhabarkaatahmad/laptop-prices-dataset-october-2023>
- [2] Tanweer Alam. 2018. A Reliable Communication Framework and Its Use in Internet of Things (IoT). 3 (05 2018).

- [3] Anonymous. 2014. Towards a Comprehensive Picture of the Great Firewall's DNS Censorship. In *IEEE Symposium on Foundations of Computational Intelligence*. <https://api.semanticscholar.org/CorpusID:8688873>
- [4] Anonymous, Arian Akhavan Niaki, Nguyen Phong Hoang, Phillipa Gill, and Amir Houmansadr. 2020. Triplet Censors: Demystifying Great Firewall's DNS Censorship Behavior. In *10th USENIX Workshop on Free and Open Communications on the Internet (FOCI 20)*. USENIX Association. <https://www.usenix.org/conference/foci20/presentation/anonymous>
- [5] Kevin Bock, Yair Fax, Kyle Reese, Jasraj Singh, and Dave Levin. 2020. Detecting and Evading Censorship-in-Depth: A Case Study of Iran's Protocol Whitelister. In *10th USENIX Workshop on Free and Open Communications on the Internet (FOCI 20)*. USENIX Association. <https://www.usenix.org/conference/foci20/presentation/bock>
- [6] Cecylia Bocovich, Arlo Breault, David Fifield, Serene, and Xiaokang Wang. 2024. Snowflake, a censorship circumvention system using temporary WebRTC proxies. In *33rd USENIX Security Symposium (USENIX Security 24)*. USENIX Association, Philadelphia, PA, 2635–2652. <https://www.usenix.org/conference/usenixsecurity24/presentation/bocovich>
- [7] Rajarshi Roy Chowdhury and Pg Emeroylariffion Abas. 2022. A survey on device fingerprinting approach for resource-constraint IoT devices: Comparative study and research challenges. *Internet of Things* 20 (2022), 100632. <https://doi.org/10.1016/j.iot.2022.100632>
- [8] Fredrik Dahlqvist, Mark Patel, Alexander Rajko, and Jonathan Shulman. 2019. Growing opportunities in the Internet of Things. *McKinsey & Company* 22 (2019).
- [9] Roger Dingledine, Nick Mathewson, and Paul Syverson. 2004. Tor: the second-generation onion router. In *Proceedings of the 13th Conference on USENIX Security Symposium - Volume 13* (San Diego, CA) (SSYM'04). USENIX Association, USA, 21.
- [10] Jason A Donenfeld. 2017. WireGuard: Next Generation Kernel Network Tunnel. In *NDSS*. 1–12.
- [11] Roya Ensafi, Philipp Winter, Abdullah Mueen, and Jedidiah R Crandall. 2014. Large-scale Spatiotemporal Characterization of Inconsistencies in the World's Largest Firewall. *arXiv preprint arXiv:1410.0735* (2014).
- [12] ESPHome. 2018. ESPHome. <https://github.com/esphome/esphome>.
- [13] Shencha Fan, Jackson Sippe, Sakamoto San, Jade Sheffey, David Fifield, Amir Houmansadr, Elson Wedwards, and Eric Wustrow. 2025. Bleeding Wall: A Hematologic Examination on the Great Firewall. *Network and Distributed System Security (NDSS)* (2025).
- [14] Benjamin Haas. 2017. Man in China sentenced to five years' jail for running VPN. *The Guardian* (Dec 2017). <https://www.theguardian.com/world/2017/dec/22/man-in-china-sentenced-to-five-years-jail-for-running-vpn>
- [15] Nguyen Phong Hoang, Arian Akhavan Niaki, Jakub Dalek, Jeffrey Knockel, Pellaeon Lin, Bill Marczak, Masashi Crete-Nishihata, Phillipa Gill, and Michalis Polychronakis. 2021. How Great is the Great Firewall? Measuring China's {DNS} Censorship. In *30th USENIX Security Symposium (USENIX Security 21)*. 3381–3398.
- [16] Amir Houmansadr, Chad Brubaker, and Vitaly Shmatikov. 2013. The Parrot Is Dead: Observing Unobservable Network Communications. In *Proceedings of the 2013 IEEE Symposium on Security and Privacy (SP '13)*. IEEE Computer Society, USA, 65–79. <https://doi.org/10.1109/SP.2013.14>
- [17] Muhammad Iqbal and Imam Riadi. 2019. Analysis of security virtual private network (VPN) using openVPN. *International Journal of Cyber-Security and Digital Forensics* 8, 1 (2019), 58–65.
- [18] Omid Jafari, Preeti Maurya, Parth Nagarkar, Khandker Mushfiqul Islam, and Chidambaram Crushev. 2021. A survey on locality sensitive hashing algorithms and their applications. *arXiv preprint arXiv:2102.08942* (2021).
- [19] Citizen Lab and Others. 2014. URL testing lists intended for discovering website censorship. <https://github.com/citizenlab/test-lists>.
- [20] Leslie Lamport. 2001. Paxos Made Simple. *ACM SIGACT News (Distributed Computing Column)* 32, 4 (Whole Number 121, December 2001) (December 2001), 51–58. <https://www.microsoft.com/en-us/research/publication/paxos-made-simple/>
- [21] R. Li and et al. 2024. DeviceRadar: Online IoT Device Fingerprinting in ISPs Using Programmable Switches. *IEEE/ACM Transactions on Networking* (2024).
- [22] Ruoyu Li, Qing Li, Tao Lin, Qingsong Zou, Dan Zhao, Yucheng Huang, Gareth Tyson, Guorui Xie, and Yong Jiang. 2024. DeviceRadar: Online IoT Device Fingerprinting in ISPs Using Programmable Switches. *IEEE/ACM Transactions on Networking* 32, 5 (2024), 3854–3869. <https://doi.org/10.1109/TNET.2024.3398778>
- [23] Petar Maymounkov and David Mazières. 2002. Kademlia: A Peer-to-Peer Information System Based on the XOR Metric. In *Revised Papers from the First International Workshop on Peer-to-Peer Systems (IPTPS '01)*. Springer-Verlag, Berlin, Heidelberg, 53–65.
- [24] R. Perdisci, T. Papastergiou, O. Alrawi, and M. Antonakakis. 2020. IoTFinder: Efficient Large-Scale Identification of IoT Devices via Passive DNS Traffic Analysis. In *2020 IEEE European Symposium on Security and Privacy (EuroS&P)*. Genoa, Italy.
- [25] Vitor Pereira, Ahmed Irfan, Vinod Yegneswaran, Nick Feamster, Prateek Mittal, and Vitaly Shmatikov. 2025. Position Paper: A Case for Machine-Checked Verification of Circumvention Systems. *Free and Open Communications on the Internet* (2025).
- [26] Tor Project. 2006. *Tor Project* (2006). <https://community.torproject.org/threat-model/threat-positioning/network/>
- [27] Ram Sundara Raman, Mona Wang, Jakub Dalek, Jonathan Mayer, and Roya Ensafi. 2022. Network measurement methods for locating and examining censorship devices. In *Proceedings of the 18th International Conference on Emerging Networking EXperiments and Technologies* (Roma, Italy) (CoNEXT '22). Association for Computing Machinery, New York, NY, USA, 18–34. <https://doi.org/10.1145/3555050.3569133>
- [28] Ram Sundara Raman, Prerana Shenoy, Katharina Kohls, and Roya Ensafi. 2020. Censored Planet: An Internet-wide, Longitudinal Censorship Observatory. In *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security* (Virtual Event, USA) (CCS '20). Association for Computing Machinery, New York, NY, USA, 49–66. <https://doi.org/10.1145/3372297.3417883>
- [29] Espressif Systems. 2025. ESP SoCs. <https://www.espressif.com/en/products/socs>
- [30] Jay Thom, Nathan Thom, Shamik Sengupta, and Emily Hand. 2022. Smart Recon: Network Traffic Fingerprinting for IoT Device Identification. In *2022 IEEE 12th Annual Computing and Communication Workshop and Conference (CCWC)*. 0072–0079. <https://doi.org/10.1109/CCWC54503.2022.9720739>
- [31] Vasilis Ververis, Tatiana Ermakova, Marios Isaakidis, Simone Basso, Benjamin Fabian, and Stefania Milan. 2021. Understanding Internet Censorship in Europe: The Case of Spain. In *Proceedings of the 13th ACM Web Science Conference 2021* (Virtual Event, United Kingdom) (WebSci '21). Association for Computing Machinery, New York, NY, USA, 319–328. <https://doi.org/10.1145/3447535.3462638>
- [32] Ryan Wails, George Arnold Sullivan, Micah Sherr, and Rob Jansen. 2024. On Precisely Detecting Censorship Circumvention in Real-World Networks. *Proceedings 2024 Network and Distributed System Security Symposium* (2024). <https://api.semanticscholar.org/CorpusID:266696753>
- [33] Zachary Weinberg, Diogo Barradas, and Nicolas Christin. 2021. Chinese Wall or Swiss Cheese? Keyword filtering in the Great Firewall of China. In *Proceedings of the Web Conference 2021* (Ljubljana, Slovenia) (WWW '21). Association for Computing Machinery, New York, NY, USA, 472–483. <https://doi.org/10.1145/3442381.3450076>
- [34] WireGuard. 2025. Known Limitations - WireGuard. <https://www.wireguard.com/known-limitations/>
- [35] Mingshi Wu, Jackson Sippe, Danesh Sivakumar, Jack Burg, Peter Anderson, Xiaokang Wang, Kevin Bock, Amir Houmansadr, Dave Levin, and Eric Wustrow. 2023. How the Great Firewall of China Detects and Blocks Fully Encrypted Traffic. In *32nd USENIX Security Symposium (USENIX Security 23)*. USENIX Association, Anaheim, CA, 2653–2670. <https://www.usenix.org/conference/usenixsecurity23/presentation/wu-mingshi>
- [36] Mingshi Wu, Ali Zohaib, Zakir Durumeric, Amir Houmansadr, and Eric Wustrow. 2025. A Wall Behind A Wall: Emerging Regional Censorship in China. In *2025 IEEE Symposium on Security and Privacy (SP)*. IEEE Computer Society, Los Alamitos, CA, USA, 1363–1380. <https://doi.org/10.1109/SP61157.2025.00152>
- [37] Project X. 2026. VLESS (XTLS Vision Seed). <https://xtls.github.io/en/config/outbounds/vless.html>
- [38] Diwen Xue, Michalis Kallitsis, Amir Houmansadr, and Roya Ensafi. 2024. Fingerprinting Obfuscated Proxy Traffic with Encapsulated TLS Handshakes. In *33rd USENIX Security Symposium (USENIX Security 24)*. USENIX Association, Philadelphia, PA.
- [39] Diwen Xue, Benjamin Mixon-Baca, ValdikSS, Anna Ablove, Beau Kujath, Jedidiah R. Crandall, and Roya Ensafi. 2022. TSPU: Russia's Decentralized Censorship System. In *Proceedings of the 22nd ACM Internet Measurement Conference*. Association for Computing Machinery, Nice, France.
- [40] Diwen Xue, Reethika Ramesh, Arham Jain, Michalis Kallitsis, J. Alex Halderman, Jedidiah R. Crandall, and Roya Ensafi. 2022. OpenVPN Is Open to VPN Fingerprinting. In *Proceedings of the 31st USENIX Security Symposium (USENIX Security 22)*. USENIX Association.